POCKET × PhoneAI: A Local Agent Network with WebMCP Control, Twin Workspaces, and Live Conversation Continuity

***Paper ID:** INL-2026-POCKET.PHONEAI.WEB.001*
***Date:** 31 August 2026*
***Lab:** ItsNotAI Labs (Dallas)*
***Thesis:** One operator machine plus a phone is a complete agent development, shipping, and control network — not a chat toy, not a set of separate screens.*

Abstract

POCKET is a host-side multi-agent operating surface. PhoneAI is its phone-native kernel and a first-class *seat*, not a skin on the owner login. This paper describes the system as it exists on an operator PC in August 2026: (1) a **network of nodes** (desk, phone, MCP, mesh, vaults, public URL, desktop) that share one work loop; (2) **WebMCP**, a diffusion catalog that turns every function, action, and task on a webpage, desktop app, or studio into an invocable tool for agents; (3) **twin minting**, in which each account materializes an encrypted file workspace, embedded model CLIs, and a Pocket-side vault on *their* PC; (4) **live conversation continuity** with Grok, Codex, and Antigravity threads that already exist on disk; (5) **streaming** of work and Antigravity UI so the phone is not a second product.

The unifying claim is negative as well as positive: **nothing is separate**. Studios are not extra destinations for humans; they are WebMCP functions agents call while they work. PhoneAI does not run a parallel Mongo stack when the host is POCKET. Antigravity conversations are not “over there”; they are listed on the same live desk the phone attaches to.

1. Problem

Coding agents (Grok CLI, Codex, Claude Code, Antigravity, OpenCode) already hold the operator’s real projects and threads. Phone UIs typically spawn an empty sandbox, ask for a password on every studio, and treat “develop,” “ship,” and “chat” as unrelated apps. The result is that work on the phone is *about* the desk instead of *on* it.

A second problem is catalog fragmentation. MCP servers, page buttons, desktop apps, and internal studios each have their own invocation style. Agents cannot “use the studio” unless a human opens `/studio/agents`. That violates the purpose of a studio: it exists so agents can develop and ship *while they work*.

A third problem is tenancy. If every seat shares the founder’s OneDrive and PATH, the system cannot scale to website signups. Each account must mint a **digital twin** on the machine that runs the host: files they can open in Explorer, CLIs embedded in that tree, an encrypted vault, and a copy of that vault into Pocket.

2. System overview

```
PhoneAI (PWA / Expo)
    ■  pair, work, life, anti, twin
    ▼
POCKET host :8787
    ■■■ live desk (Grok / Codex / Antigravity SQLite threads)
    ■■■ WebMCP catalog + use_action
    ■■■ twin mint (tenants/<user>/{files,bin,agents,vault,pocket_vault})
    ■■■ GO plane (one sync of every live surface)
    ■■■ MCP stdio + HTTP stream
    ■■■ network develop/ship (functions, not only HTML)
```

Empirical host snapshot on the reference machine: 146 first-class agents, MCP bundle, mesh disk, PhoneAI seat `phoneai` distinct from owner `pocket`.

3. Seats and twin minting

Owner credentials remain in `~/.pocket/ACCESS.txt`. PhoneAI is user `phoneai` with its own ACCESS file and tenant root `~/.pocket/tenants/phoneai`. Website signups (`/join`) call `register()` which now **mints a twin**:

| Path | Role | |------|------| | `files/`, `twin/` | Explorer + working tree | | `bin/` | Shims for Grok, Codex, Claude, Gemini, OpenCode, Copilot, … | | `agents/` | PhoneAI-created agents (JSON). Run with `PATH=bin` | | `vault/` | Encrypted notes (`hmac-sha256-ctr-v1`) | | `pocket_vault/` and `~/.pocket/vaults/<user>/` | Same envelopes copied into Pocket | | `OPEN.cmd` | Opens *their* folder on the PC |

This is the scale path: one directory tree per account, no founder disk, CLIs inside the workspace so agents talk to PhoneAI without a global install.

Encryption is a stream cipher keyed by PBKDF2(user, `POCKET_TWIN_SECRET`). It is not a substitute for TLS on the public URL; it is at-rest isolation of vault blobs so a raw directory listing is not plaintext work.

4. WebMCP: one catalog, real invoke

WebMCP diffuses:

- fusion (UIA + OCR + vision IR nodes)
- desktop apps
- PhoneAI typed tools
- engine uses
- host MCP tools
- **work functions** (develop, ship, twin mint/open/vault/agent, Antigravity new/send/continue/read, GO sync, scan)

`use_action(name, prompt)` actually runs those invokes. Doctrine in the catalog: *Nothing is separate. Studios, twin, Antigravity, GO are WebMCP functions agents invoke while they work.*

That is how PhoneAI uses “other apps”: it does not embed every vendor SDK. It lists the live UI (fusion) and the host functions (work), then clicks or POSTs. Streaming (`/v1/phoneai/work/stream`, Antigravity read on an interval, MCP JSON-RPC stream) keeps the phone on the same conversation the PC already has.

5. Live conversation continuity

Grok threads come from `~/.grok/sessions/**/summary.json` and resume with `grok --resume`. Codex rollouts live under `~/.codex/sessions`. Antigravity conversations are SQLite trajectories under `~/.gemini/antigravity/conversations/*.db` (protobuf payloads). POCKET now lists those databases: cascade id, worktree cwd, step count, printable snippet. PhoneAI Work’s thread picker includes `antigravity_threads`. Sending from the phone still uses UI automation (clipboard, Enter, named Send) plus fusion scan — the only stable headless path until Antigravity exposes an official resume CLI.

6. GO plane

`go()` already synced keep, work_mode, long workflows, power, clouds. It now also marks `phoneai`, `twin`, `webmcp`, `network`, and `antigravity` surfaces. Agents call WebMCP **GO plane** instead of a human visiting

five URLs.

7. Streaming

- Job log tails (`stream_util`) for Codex/Grok on the desk
- SSE `/v1/phoneai/work/stream` for the phone twin
- Antigravity thread poll (`anti_read`)
- MCP protocol stream pages

Together they are one idea: the phone consumes the same running work, not a replay.

8. What more MCP + streaming can do (honest next)

Implemented: catalog + invoke for work functions; fusion clicks; SSE work; Anti read.

High leverage and not yet productized:

1. **MCP resource subscriptions** for twin files and vault envelopes so other apps (Claude Code, Cursor, OpenCode) see PhoneAI agents as resources, not URLs. 2. **SSE of fusion deltas** (new buttons/notifications) so PhoneAI can Continue without a full scan. 3. **Antigravity protobuf schema** so thread *messages* stream as text, not only OCR/string harvest. 4. **Per-seat MCP stdio** spawned with `PATH=twin/bin` so third-party agents automatically use embedded CLIs. 5. **GO as an MCP tool** with a single `go` name in the pocket MCP server (it is on WebMCP; stdio list should match).

9. Security and limits

- Public paths for PhoneAI/twin/network are intentional on LAN; Cloudflare still gates `/desk` APIs by session.
- Vault encryption is symmetric and host-local; compromise of the host secrets file recovers vaults.
- Twin mint on a shared host is a jail (`tenants/<user>`), not a VM.
- Antigravity ingest is best-effort string extraction from undocumented SQLite.
- WebMCP `use_action` is powerful; it should stay behind the same receipts as `/api/execute` for market seats (partial today).

10. Conclusion

POCKET is a local agent platform. PhoneAI is its kernel seat. The network is one work loop: mint a twin, embed CLIs, list every action, let agents develop and ship *as tools*, and continue the conversations already open in Grok, Codex, and Antigravity. Separate screens were a staging error. The product is the catalog agents call.

References (in-repo)

- `docs/research/POCKET_PLATFORM_PAPER.md`
- `docs/STUDIOS.md`
- `docs/SUBAGENT_MESH.md`
- PhoneAI README: https://github.com/ItsNotAILABS/PhoneAI
- WebMCP module `pocket.webmcp`

- Twin mint `pocket.twin_mint`
- Live desk `pocket.live_desk`